

课程笔记

CS336 Lecture 17: Policy Gradient Mechanics and GRPO

基于课程字幕与讲义脚本整理

2025 年 3 月 26 日



视频作者/频道: CS336 / Stanford

发布日期: 2025-03-26

视频时长: 约 31 分钟

视频链接: 未填写

目录

1 引言：为什么要讲 policy gradient	2
2 语言模型里的 RL 设定	2
2.1 state, action, reward	2
2.2 为什么 outcome reward 很自然	3
2.3 目标函数	4
3 Policy Gradient 的基本推导	4
3.1 从期望奖励到 log-derivative trick	4
3.2 naive policy gradient	5
4 为什么要 baseline	5
4.1 高方差问题	5
4.2 baseline 不改变目标	5
4.3 方差为什么会下降	6
4.4 advantage function	6
5 GRPO 的直觉	6
5.1 为什么是 group relative	6
5.2 和 PPO 的关系	7
6 一个可训练的 toy problem	8
6.1 排序任务	8
6.2 模型结构	9
6.3 采样和打分	9
7 从代码看实现	9
7.1 compute_deltas: 把 reward 变成可训练信号	9
7.2 compute_loss: ratio, clipping, and conservative updates	10
7.3 compute_kl_penalty: 保持和 reference model 的距离	10
7.4 训练循环的职责边界	11
8 delta 的几种选择	11
9 loss 的几种形式	12
9.1 naive loss	12
9.2 old policy ratio	12
9.3 clipping	12
9.4 KL penalty	12
10 训练流程	12
10.1 GRPO 风格的循环	12
10.2 实验现象	13

11 局限、失败模式与相关机制	13
11.1 为什么这些方法会卡住	13
11.2 一个 worked example: group mean 怎么决定更新方向	14
11.3 GRPO 和相关 alignment 方法的关系	14
12 总结	15
12.1 这一讲的结论	15
12.2 最后的提醒	15
12.3 参考	15

1 引言：为什么要讲 policy gradient

这一讲是本课程的最后一讲，由于前一讲已经概览了 *reinforcement learning from verifiable rewards* 和策略梯度类方法，这一讲不再引入全新框架，而是把 policy gradient 和 GRPO 的机制掰开揉碎，重点看三个问题：

1. 在语言模型里，RL 的 **state**、**action**、**reward** 到底对应什么；
2. 为什么 naive policy gradient 会有高方差，baseline 和 advantage 为什么能缓解它；
3. GRPO 在代码里到底怎么跑，为什么要用 group 内平均奖励做比较。

这一讲的核心

- 如果你能度量，就能优化，但 RL 的难点在于度量很稀疏、很噪声。
- 策略梯度本身不神秘，本质上就是对 $\log \pi(a | s)$ 乘上一个奖励信号。
- baseline 的作用是降方差，不是改变优化目标。

为什么这一讲很像“工程课”

因为 policy gradient 的关键问题不是能不能写公式，而是如何把公式变成一个在 ‘verifiable reward’ 下能稳定训练的系统。只要 reward 稍微稀疏一点，方差、采样、baseline、clipping、reference model 都会立刻变成工程问题。

2 语言模型里的 RL 设定

2.1 state, action, reward

把整段 response 记作一个动作 a ，语言模型 RL 的对应关系非常直接：

- **State** s ：prompt 加上已经生成的 response 前缀。
- **Action** a ：生成下一个 token，或者在 outcome reward 视角里，直接把整段 response 看成一个动作序列。
- **Reward** $R(s, a)$ ：整段 response 的好坏，本文只讨论 **outcome reward** 和 **verifiable reward**。

字幕里强调了一点：在语言模型场景中，transition dynamics 基本就是拼接操作。你做出的下一状态就是

$$s' = s + a$$

也就是把 token 继续 append 到上下文里。这和机器人控制非常不同，后者的状态由物理世界决定，语言模型的状态则更像是“自己写出来的中间草稿”。

语言模型里的 state 很“虚”

在 robotics 中，state 可能是关节角、速度、图像；在 LM 中，state 只是 token 序列。这个差别意味着：

- LM 可以自由构造 scratchpad；
- LM 可以在 test time 做 planning / search；
- 但 reward 往往更稀疏，正确与错误的边界也更硬。

process reward 不是白送的

如果能在中间步骤给出可信的 reward，训练通常会更好；但在语言模型里，“中间步骤是否对”往往比最终答案更难判断。因此本讲先聚焦 outcome reward，把机制讲清楚。

2.2 为什么 outcome reward 很自然

这一讲选择 outcome reward，不只是因为它简单，而是因为它和课程脚本里的 toy task 完全一致：你给定一个 prompt，模型生成一个完整 response，最后用一个确定性的函数判断它是否“做对了”。

这类 reward 的优点是实现成本低，且容易复现：

- 你可以直接写一个 Python 函数来算分；
- 训练和评估使用同一个标准；
- 不需要额外训练 reward model；
- 很适合数学、代码、排序、格式控制这类任务。

课程里为什么先讲这种 reward

因为它最能暴露 policy gradient 的本质问题：reward 稀疏、方差大、更新不稳定。只要把这类 task 讲明白，后面更复杂的 RLHF / RLAIIF 也更容易理解。

reward 类型	优点	缺点
outcome / verifiable reward	可自动计算，标准明确	反馈稀疏，难学
process reward learned model	中间步骤也能给分 能吸收人类偏好	需要判断过程本身是否可信 额外系统复杂度高

表 1: 为什么课程这里先采用 outcome reward：它最利于把机制讲透

2.3 目标函数

如果把 prompt 视作由环境给出的 s , response 视作 policy 采样出来的 a , 那么目标就是最大化期望奖励:

$$\max_{\pi} \mathbb{E}_{s \sim p(s), a \sim \pi(\cdot|s)} [R(s, a)].$$

在 outcome reward 设定下, reward 只在整段输出结束后给出, 因此 discounting、bootstrapping 这些经典 RL 概念在这里不那么核心。

为什么这里几乎不谈 discount

如果一个 response 的好坏只能在最后统一打分, 那么 reward 的主要难点已经不是时间折扣, 而是稀疏性和方差。也就是说, 什么时候给分不如怎么把这个分数变得可学更重要。

3 Policy Gradient 的基本推导

3.1 从期望奖励到 log-derivative trick

为了记号简洁, 把整段 response 当作一个整体动作 a 。那么

$$\mathbb{E}[R] = \int p(s) \pi(a | s) R(s, a) ds da.$$

对 policy 参数求梯度:

$$\nabla \mathbb{E}[R] = \int p(s) \nabla \pi(a | s) R(s, a) ds da = \int p(s) \pi(a | s) \nabla \log \pi(a | s) R(s, a) ds da.$$

于是得到熟悉的形式:

$$\nabla \mathbb{E}[R] = \mathbb{E}[\nabla \log \pi(a | s) R(s, a)].$$

如果把 response 展开成 token 序列 $a_{1:T}$, 那 policy 其实在每一步都给出一个条件分布:

$$\pi(a_{1:T} | s) = \prod_{t=1}^T \pi(a_t | s, a_{<t}),$$

所以

$$\log \pi(a_{1:T} | s) = \sum_{t=1}^T \log \pi(a_t | s, a_{<t}).$$

这件事非常重要, 因为它说明 **sequence-level 的 reward 最后会分摊到每一个 token 的 log prob 上**。虽然最后只给一个分数, 但梯度会回流到整条生成轨迹。

policy gradient 的直觉

- 采到好样本, 就提高它的概率;
- 采到差样本, 就降低它的概率;
- 变化的幅度由 reward 决定。

为什么要对 log prob 求梯度

‘log’ 把乘法关系变成加法, 梯度也更稳定。更重要的是, 它让 “提高一个 sample 的概率” 变成一个直接可微的信号, 便于把 reward 接到反向传播里。

3.2 naive policy gradient

最朴素的做法就是：

1. 采样一个 prompt s ;
2. 从当前 policy 采样 response a ;
3. 用 $\nabla \log \pi(a | s) R(s, a)$ 更新参数。

这和 supervised fine-tuning 很像，只不过监督数据不是人工标注，而是模型自己采样出来后再按 reward 过滤和加权。

稀疏奖励会把更新打空

如果 $R \in \{0, 1\}$ ，多数样本 reward 都是 0，那么很多 batch 的梯度就是近似 0。模型如果一开始就很差，可能连“碰到正奖励样本”的机会都很少。

4 为什么要 baseline

4.1 高方差问题

在语言模型 RL 中，reward 往往是稀疏的、延迟的、并且分布差异很大。字幕里的 toy 例子把这个问题讲得很直白：

- state s_1 的 reward 可能是 11 或 9;
- state s_2 的 reward 可能是 0 或 2;
- 直接拿 reward 做更新，会出现“9 比 2 大，所以看起来更值得强化”的局部错觉。

但是从全局看， s_2 明显更难、reward 更低，不能只看绝对值。这里真正要比较的是“**这个 action 比在当前 state 下的平均水平好多少**”。

4.2 baseline 不改变目标

引入 baseline $b(s)$ 后，我们优化的是

$$\mathbb{E}[R - b(s)].$$

只要 baseline 不依赖动作 a ，这个变换不会改变原来的最优解，因为

$$\mathbb{E}[b(s)] = \int p(s) b(s) ds$$

与 policy 无关，因而只是加了一个常数项。

baseline 的要求

baseline 可以依赖 state，不能依赖 action。直观上，它是“当前 state 下你本来就该得到多少”的参照线。

state	action 1	action 2	直觉
s_1	11	9	两个都还不错，但 11 更好
s_2	0	2	2 仍然优于 0，但绝对值更低

表 2: 字幕里的两状态例子：关键不在绝对 reward，而在当前 state 下的相对提升

baseline 的数学意义

如果 baseline 只依赖 state，那么

$$\mathbb{E}_{a \sim \pi(\cdot | s)} [\nabla \log \pi(a | s) b(s)] = 0.$$

因此我们可以安全地把 reward 改成 $R - b(s)$ ，而不改变期望梯度的方向。

4.3 方差为什么会下降

在 toy 例子里，原始 reward 的方差大约是 5.3；如果对两个 state 分别取 baseline 10 和 1，方差会降到约 1.1。这个例子想说明的不是某个数值，而是一个原则：

baseline 的作用

- 保留期望优化方向；
- 压缩无用的绝对量级差异；
- 让梯度更新更稳定、更快收敛。

4.4 advantage function

经典 RL 里有：

$$V(s) = \mathbb{E}[R | s], \quad Q(s, a) = \mathbb{E}[R | s, a],$$

以及 advantage：

$$A(s, a) = Q(s, a) - V(s).$$

在这一讲的 outcome reward 设定里， Q 和 R 基本可以视为同一个量，因此当 baseline 取成

$$b(s) = \mathbb{E}[R | s]$$

时， $R - b(s)$ 就是 advantage 的形式。

难点不在公式，而在估计

理论上最优 baseline 可以写出 closed form，但实际里很难精确计算期待值，所以大家通常用样本平均、group mean，或者 value model 去近似它。

5 GRPO 的直觉

5.1 为什么是 group relative

GRPO 的关键不在于名字，而在于它利用了语言模型的自然分组结构：

- 同一个 prompt 可以采样出多条 response;
- 这些 response 天然构成一个 group;
- group 内平均 reward 就是一个很自然的 baseline。

这就是“relative”的来源。它比较的是“同一 prompt 下，谁比同组其他 response 更好”。

为什么 GRPO 适合语言模型

对同一个 prompt 采样多条 response，本来就是 LM 训练里最自然的采样单位，因此 baseline 不必再从全局状态空间拟合，而可以在 group 内做相对比较。

方法	信号来源	baseline / critic	特点
REINFORCE	直接用 reward	无显式 critic	最朴素，方差大
PPO	reward + value estimate	需要 critic / value model	更稳定，但系统更重
GRPO	group 内相对 reward	group mean 近似 baseline	适合多样本 response 的 LM 场景

表 3: 三种常见策略梯度风格方法的关系：GRPO 可以看成是 LM 场景下更轻量的 PPO 变体

为什么 GRPO 不一定要 critic

critic 的作用是估计 state value，帮助你减少方差。但在同一个 prompt 下采样多个 response 时，group mean 本身就已经提供了一个很自然的、低成本的 baseline 近似，因此可以先不用单独训练 value network。

group mean 也不是万能的

group mean 是一个很强的经验 baseline，但它并不等于最优 baseline。若 group 太小，估计会噪声大；若 group 太大，采样和计算成本会上升。课程脚本用的是一个教学上很好理解的折中。

5.2 和 PPO 的关系

课程里把 GRPO 描述成一种 **不需要 critic** 的 PPO 风格简化版。你仍然会看到：

- policy ratio;
- clipping;
- 参考策略或旧策略;
- KL regularization。

但 group 结构让 baseline 近似得更自然，所以实现上可以更轻。

6 一个可训练的 toy problem

6.1 排序任务

为了能在本地跑通，讲义脚本构造了一个很小的 RL 任务：给定一组数字，模型要输出排序后的序列。

- Prompt：一组数字；
- Response：希望是排序后的数字；
- Reward：衡量 response 和排序答案接近的程度。

这里故意不直接用“完全正确 = 1，否则 0”的极端奖励，而是设计了两个版本：

1. **位置匹配奖励**：response 与 ground truth 在多少位置上一致；
2. **inclusion + ordering 奖励**：既看 token 是否出现，也看局部顺序是否合理。

为什么要给 partial credit

如果只给 0/1 奖励，大量样本都没有学习信号。给 partial credit 可以让模型在“还没完全做对”时也能得到有用反馈。

函数	直觉	作用
sort_distance_reward	距离 ground truth 多近	给连续一点的反馈
sort_inclusion_ordering_reward	是否包含正确元素且顺序合理	更像真实排序任务

表 4: 脚本里两种 reward 的教学意图：一个更连续，一个更接近排序语义

```
1 def reward(prompt, response):
2     sorted_gt = sorted(prompt)
3     score_match = how_many_positions_match(response, sorted_gt)
4     score_order = how_well_ordered(response, sorted_gt)
5     return score_match + 0.5 * score_order
```

Listing 1: toy reward pseudocode

这个 toy reward 的目标不是严格复刻真实排序，而是让训练信号足够连续。因为如果 reward 只有“全对才给 1 分”，那模型在早期几乎看不到梯度；而当一个 response 已经接近正确时，partial credit 可以继续推动它往正确方向移动。

prompt	response	现象	分数趋势
[3,1,0,2]	[0,1,2,3]	完全排序正确	高
[3,1,0,2]	[0,3,1,2]	元素都在，但顺序差	中
[3,1,0,2]	[7,2,2,5]	元素错且顺序也差	低

表 5: 课程脚本里的两个 reward 函数都想表达同一件事：不要只有“对/错”，还要能区分“差一点”和“差很多”

为什么这些样例很重要

toy task 的价值在于，你能把 reward 函数、采样、baseline 和 policy update 放在同一张表里看它们如何相互影响。真实模型里这些细节被 transformer 和大规模数据掩盖了，但机制是同一套。

6.2 模型结构

脚本里的 toy model 很小，但足够展示 policy gradient 的机制：

- prompt 和 response 长度固定；
- 每个位置有独立的 encode/decode 参数；
- 最后通过 embedding weight 生成 logits。

这不是一个真实的 transformer，而是一个便于分析的最简模型。

6.3 采样和打分

训练流程大致如下：

1. 输入 prompt；
2. 从当前模型采样多个 responses；
3. 用 reward function 对每条 response 打分；
4. 根据 delta 形式构造更新信号；
5. 用 policy gradient 更新模型。

7 从代码看实现

这一节专门把脚本中的核心函数拆开。这样做的目的不是为了复读代码，而是把“公式怎么落地”讲完整。

7.1 compute_deltas: 把 reward 变成可训练信号

脚本里的 δ 不是单纯的 reward。它更像是一个 **update signal**，负责告诉优化器应该强化谁、压低谁。四种模式对应四种不同的信号整形方式：

```
1 if mode == "rewards":
2     return rewards
3
4 if mode == "centered_rewards":
5     mean_rewards = rewards.mean(dim=-1, keepdim=True)
6     return rewards - mean_rewards
7
8 if mode == "normalized_rewards":
9     mean_rewards = rewards.mean(dim=-1, keepdim=True)
```

```

10     std_rewards = rewards.std(dim=-1, keepdim=True)
11     return (rewards - mean_rewards) / (std_rewards + 1e-5)
12
13 if mode == "max_rewards":
14     max_rewards = rewards.max(dim=-1, keepdim=True)[0]
15     return torch.where(rewards == max_rewards, rewards, torch.zeros_like(rewards))

```

Listing 2: compute_deltas: reward to update signal

这段代码的本质

它没有改变 reward 的语义，只是在做“局部 baseline + 尺度控制 + winner emphasis”。这也是为什么 centered 和 normalized 往往比 raw rewards 更稳定。

7.2 compute_loss: ratio, clipping, and conservative updates

如果只做最朴素的 policy gradient，那么 loss 就是

$$\mathcal{L} = -\mathbb{E}[\log \pi(a | s) \delta].$$

但为了更稳定，脚本引入了 old policy。此时就有 ratio：

$$r = \frac{\pi_{\theta}(a | s)}{\pi_{\text{old}}(a | s)}.$$

```

1  if mode == "naive":
2      return -einsum(log_probs, deltas, "...").mean()
3
4  if mode == "unclipped":
5      ratios = torch.exp(log_probs - old_log_probs)
6      return -einsum(ratios, deltas, "...").mean()
7
8  if mode == "clipped":
9      epsilon = 0.01
10     ratios = torch.exp(log_probs - old_log_probs)
11     unclipped = einsum(ratios, deltas, "...")
12     clipped = einsum(torch.clamp(ratios, 1 - epsilon, 1 + epsilon), deltas, "...")
13     return -torch.minimum(unclipped, clipped).mean()

```

Listing 3: compute_loss: naive / unclipped / clipped

为什么 clipped 更像“保守更新”

如果 ratio 变得太大，某些 response 会对梯度产生异常强的影响。clipping 的作用就是把这种放大效应截断，让单次 update 不至于把模型推得太远。

7.3 compute_kl_penalty: 保持和 reference model 的距离

除了 old policy，脚本还允许加入 reference model 的 KL 惩罚。它的作用不是替代 reward，而是作为“别漂太远”的约束。

```

1 return (torch.exp(ref_log_probs - log_probs)
2         - (ref_log_probs - log_probs)
3         - 1).sum(dim=-1).mean()

```

Listing 4: compute_kl_penalty

这一项的直觉很简单：如果 current policy 跟 reference model 越来越不像，就会付出额外代价。这样可以避免模型在追 reward 时把原来的语言能力和风格能力一起丢掉。

7.4 训练循环的职责边界

从系统角度看，这个训练 loop 实际上是在做四件互不混淆的事：

- **sample**: 用当前 policy 采样 responses;
- **score**: 用确定性的 reward 函数打分;
- **freeze**: old policy / reference model 保持固定;
- **update**: 只更新 current policy。

这套分工很重要，因为它让“谁在提供信号、谁在被更新”保持清楚。只要这条边界混了，GR-PO/PPO 这类方法就会变得很难调。

8 delta 的几种选择

脚本里把用于更新的量统一记成 δ ，然后尝试了几种常见做法：

- **rewards**: 直接用 raw rewards;
- **centered rewards**: 减去 group mean;
- **normalized rewards**: 再除以标准差;
- **max rewards**: 只保留组内最大 reward。

最常见的是 centered rewards

在同一个 prompt 的多个 response 里，减掉 group mean 是最自然的 baseline 近似。它对应“相对同组平均表现的好坏”。

模式	公式	教学含义
rewards	$\delta_i = r_i$	直接用绝对 reward，最朴素但方差最大
centered_rewards	$\delta_i = r_i - \bar{r}$	组内相对比较，最像 baseline
normalized_rewards	$\delta_i = (r_i - \bar{r})/(\sigma + \epsilon)$	再压缩尺度，训练更平滑
max_rewards	只保留组内最大值	强调 winner-take-all 的更新方向

表 6: 脚本中四种 δ 选择的差别：本质上都是在决定“谁应该被强化”

9 loss 的几种形式

9.1 naive loss

最直接的 loss 是

$$\mathcal{L}_{\text{naive}} = -\mathbb{E}[\log \pi(a | s) \delta].$$

它就是把 log probability 和 reward 信号直接乘起来。

9.2 old policy ratio

如果引入旧策略 π_{old} ，则会出现 ratio：

$$\frac{\pi(a | s)}{\pi_{\text{old}}(a | s)}.$$

脚本里把它写成 log prob 差：

$$\exp(\log \pi - \log \pi_{\text{old}}).$$

旧策略必须 freeze

‘old policy’ 只能当常量用，不能让梯度穿过去。否则 ratio 的意义会被破坏，整个优化会偏掉。

9.3 clipping

和 PPO 类似，ratio 太大或太小时都不稳定，所以会做 clipping：

$$r = \frac{\pi}{\pi_{\text{old}}}, \quad \tilde{r} = \text{clip}(r, 1 - \epsilon, 1 + \epsilon).$$

最后用 unclipped 和 clipped 的较小者构成保守更新。

9.4 KL penalty

如果担心 RL 把模型带偏太远，还可以加 reference model 的 KL penalty：

$$\text{KL}(p \| q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right].$$

脚本里用了一个数值上可算的估计：

$$\mathbb{E}_p \left[\frac{q}{p} - \log \frac{q}{p} - 1 \right].$$

KL penalty 的作用

它是“别忘了原来的模型能力”的安全带。你想让模型获得新能力，但又不想把旧能力完全冲掉。

10 训练流程

10.1 GRPO 风格的循环

脚本中的训练循环可以概括为：

1. 采样一批 prompts;
2. 对每个 prompt 采样多个 responses;
3. 计算 rewards 和 deltas;
4. 可选地计算 reference model 的 log probs;
5. 用当前 policy 的 log probs 算 loss;
6. 反向传播, 更新参数。

10.2 实验现象

课程脚本尝试了三种设置:

- raw rewards;
- centered rewards;
- normalized rewards。

结论是:

- raw rewards 往往学得慢, 容易卡住;
- centered rewards 更稳定一些;
- normalized rewards 不一定有明显增益, 甚至可能引入不必要的偏差;
- 这类 toy RL 任务本身就很容易卡在局部最优。

设置	观察	含义
raw rewards	更新噪声大, 训练容易停滞	绝对 reward 直接进梯度, 方差过高
centered rewards	学习更稳定, 负样本也有作用	相对比较比绝对分数更有信息
normalized rewards	有时接近 centered 的效果	标准差缩放主要是稳定尺度

表 7: 三种实验设置的经验结论: GRPO 这类方法的关键通常是“先把信号做稳”

RL 不是“设个 reward 就会自动学会”

即便任务和 reward 都写得很清楚, policy gradient 仍然可能因为方差、初始化和超参问题而学不动。

11 局限、失败模式与相关机制

11.1 为什么这些方法会卡住

这一讲的 toy 系统其实已经暴露了策略梯度的几个典型失败模式:

- **reward 稀疏**: 大多数 sample 没信号，更新太慢；
- **baseline 不准**: group 太小或方差太大时，centered signal 仍然不稳；
- **更新过猛**: ratio 太大，模型可能一下子偏离原策略；
- **KL 太弱**: 模型会为了 reward 牺牲原本的语言能力；
- **KL 太强**: 模型几乎不动，reward 学不上去。

问题	表面现象	本质原因
稀疏 reward	训练曲线长时间不动	梯度大多数时候接近 0
group 太小	centered 后仍很抖	baseline 估计噪声大
ratio 太大	loss 突然震荡	单步更新过猛
KL 太弱	语言质量下降	目标函数只看 reward, 不看漂移
KL 太强	reward 学不起来	约束压过了优化

表 8: GRPO / PPO 风格训练最常见的失败模式：问题通常不在“公式错了”，而在“信号和约束没平衡好”

11.2 一个 worked example: group mean 怎么决定更新方向

设同一个 prompt 采到三个 response, 它们的 reward 是 $[3, 2, 1]$ 。那么 group mean 是 2, centered rewards 就是 $[+1, 0, -1]$ 。这意味着：

- 第一条 response 被强化；
- 第二条 response 基本不动；
- 第三条 response 被压低。

如果把它换成 normalized rewards, 结果只是再除以一个标准差；更新方向不变，主要变化是步幅更统一。

11.3 GRPO 和相关 alignment 方法的关系

如果把这门课放到更大的对齐框架里看，GRPO 和 RLHF、PPO、REINFORCE 的关系可以总结成一句话：它们共享同一个 policy gradient 骨架，只是在 baseline、约束和奖励来源上做了不同选择。

方法	reward 来源	约束	适用感受
REINFORCE	直接 reward	通常没有显式约束	最基础，但方差大
PPO	reward model 或环境 reward	clipping + value baseline	经典、稳定、但组件多
GRPO	group-relative reward	clipping + KL, 不一定 要 critic	对 LM response group 很自然
RLHF 训练链路	人类偏好或奖励模型	取决于实现	更偏系统工程

表 9: 把 GRPO 放到更大的 alignment 图景里：它不是另起炉灶，而是把现有 policy gradient 组件重新组合

这一节想传达的判断

GRPO 的价值不是“发明了一个完全不同的 RL”，而是它把 *group baseline*、*clipping*、*KL* 这些已经成熟的稳定化技巧放进了语言模型最自然的采样结构里，因此实现上更轻，直觉上也更顺。

12 总结

12.1 这一讲的结论

四个核心 takeaway

1. 在 LM 里，RL 的 state/action/reward 定义很自然，但 reward 往往非常稀疏。
2. Naive policy gradient 的形式简单，但方差很大。
3. Baseline 和 advantage 的本质是降方差，不是改目标。
4. GRPO 利用了同一 prompt 下多个 response 的 group 结构，所以在 LM 场景里特别合适。

12.2 最后的提醒

这一讲其实在传达一个很朴素但重要的判断：

如果你能度量某个东西，就可以把它塞进 optimization loop 里，但真正困难的是让这个度量足够稳定、足够密、足够有用。

这也是为什么 policy gradient 在语言模型里既强大又麻烦：它给了你直接优化目标的入口，也把训练稳定性问题完整暴露了出来。

12.3 参考

- Schulman et al., Proximal Policy Optimization Algorithms
- Deep RL textbook / CS224R policy gradient notes
- GRPO / verifiable rewards 相关课程材料